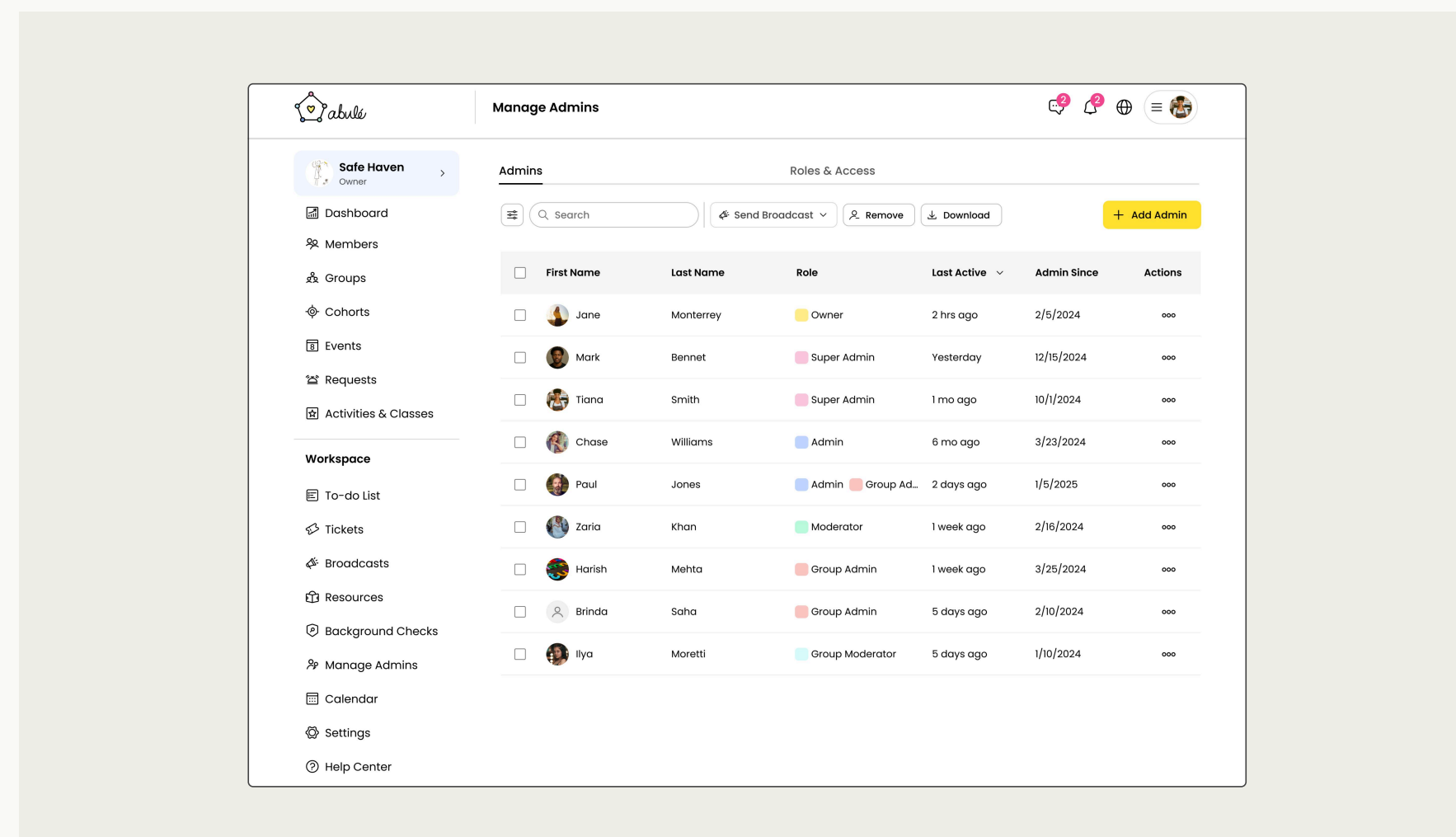

abulé
Manage Admins
Safe Haven
Owner
Dashboard
Members
Groups
Cohorts
Events
Requests
Activities & Classes
Workspace
To-do List
Tickets
Broadcasts
Resources
Background Checks
Manage Admins
Calendar
Settings
Help Center
Admins
Roles & Access
Search
Send Broadcast
Remove
Download
+ Add Admin

| First Name | Last Name | Role | Last Active | Admin Since | Actions |
|---|---|---|---|---|---|
| Jane | Monterrey | Owner | 2 hrs ago | 2/5/2024 | ••• |
| Mark | Bennet | Super Admin | Yesterday | 12/15/2024 | ••• |
| Tiana | Smith | Super Admin | 1 mo ago | 10/21/2024 | ••• |
| Chloe | Williams | Admin | 6 mo ago | 2/23/2024 | ••• |
| Paul | Jones | Admin, Group Ad... | 2 days ago | 1/5/2026 | ••• |
| Daria | Khan | Moderator | 1 week ago | 2/16/2024 | ••• |
| Harish | Mehta | Group Admin | 1 week ago | 2/26/2024 | ••• |
| Brinda | Sahu | Group Admin | 5 days ago | 2/26/2024 | ••• |
| Aya | Moretti | Group Moderator | 5 days ago | 1/10/2024 | ••• |

# Role-based Admin Portal Design for *Abulé*

## Role & Focus

- 0-to-1 Product Strategy & Design
- Platform Architecture
- Information Architecture
- Design System Expansion
- MVP Roadmap & Delivery
- Phased Roll-out Strategy
- Cross-functional Collaboration

## Problem

- Abulé's web app was ready for launch but operational control and oversight was missing across three user groups.

## Solution & Impact

- I architected Abulé's dual-portal admin ecosystem and designed the Community Admin Portal, a role-based system that enables community creators to manage billing, permissions, and operations independently.
- Architectural decision saved ~25% engineering effort by avoiding duplicate systems
- Delivered a 6-level role system with 65+ screens, that supports everyone from solo owners to large admin teams without complex structural changes

## Tools

- Figma • Figjam • Linear • Notion • MS Teams

## Timeline

- 2 months- Delivered launch-ready designs and documentation

---

TL;DR
Context
Defining platform architecture
Building The IA
What I Designed
Outcome

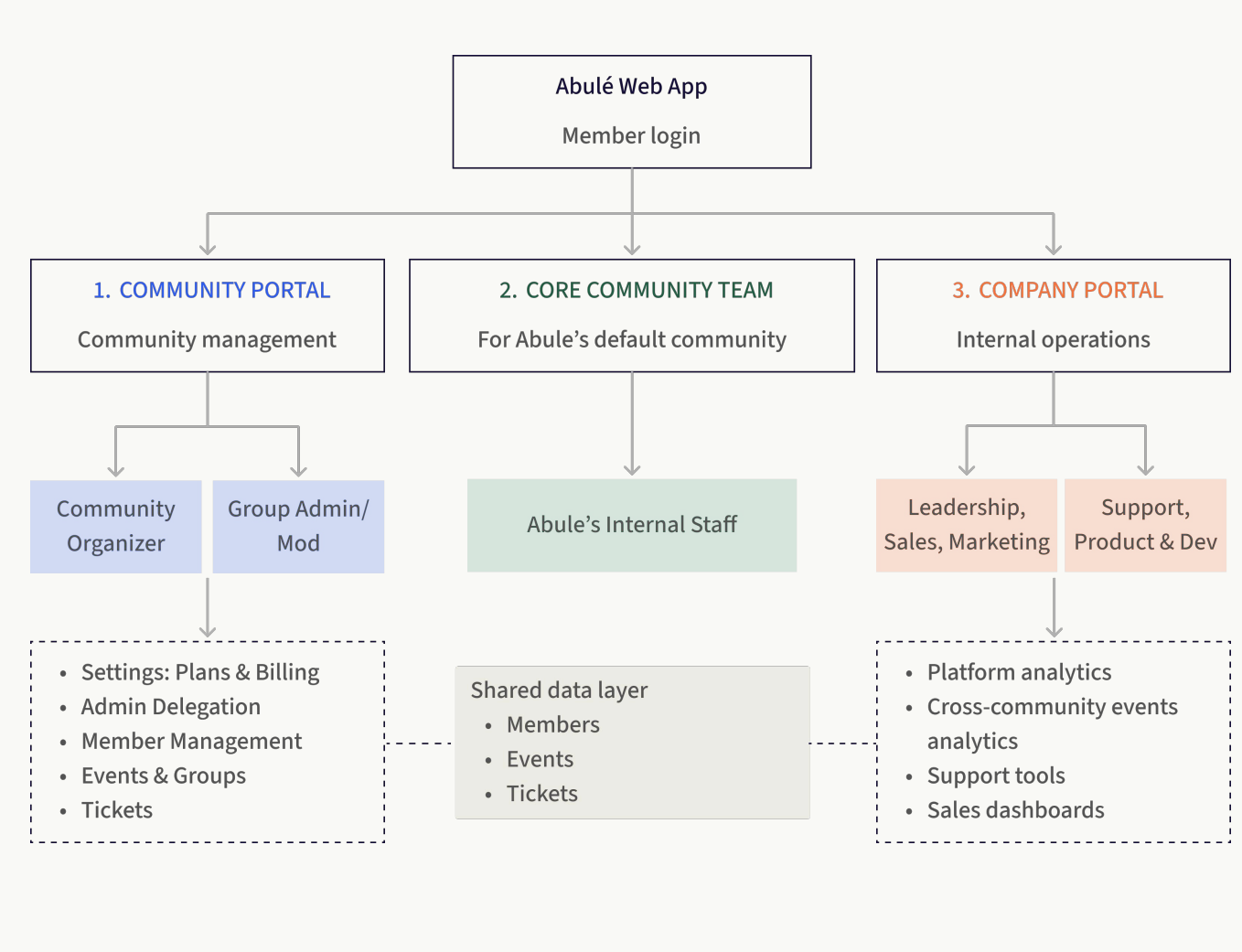
CONTEXT

## What's Abulé?

Abulé is a seed-stage platform that helps people find, manage, and provide care within communities. Schools, companies, or nonprofits can use it to create communities where members can organize and manage support.

## Key Constraints

- 2-month timeline to deliver a functional foundation
- Limited engineering bandwidth due to existing web-app delays
- Pre-launch product with no direct user access


Abulé Web App
Member login
1. COMMUNITY PORTAL
Community management
2. CORE COMMUNITY TEAM
For Abule's default community
3. COMPANY PORTAL
Internal operations
Community Organizer
Group Admin/ Mod
Abule's Internal Staff
Leadership, Sales, Marketing
Support, Product & Dev
Settings: Plans & Billing
Admin Delegation
Member Management
Events & Groups
Tickets
Shared data layer
Members
Events
Tickets
Platform analytics
Cross-community events analytics
Support tools
Sales dashboards

> Three user groups with overlapping tasks. Without an admin portal, communities would be unmanageable, billing would be manual. Internal teams would have no visibility. Fragmented workflows would block revenue & scale. To launch we needed two things: Settings (plans, billing, configuration) and Admin Management.

DEFINING PLATFORM ARCHITECTURE

The PRD established that we needed two separate admin environments:
Community Portal for community organizers to manage their own communities.
Company Portal for Abulé's internal teams to manage platform-wide operations.

## But the question was- Where does the Core Community Team Fit?

The founder and lead engineer thought the Core Community Team should be in the Company Portal since they're Abulé employees. But when I mapped their daily tasks, it was clear they worked like community managers, creating events, moderating members, and managing groups. I built a side-by-side comparison of both options and brought it to the team.

| with Community Portal | with Company Portal |
|---|---|
| ✓ 100% task alignment with community admins. | ✓ Keeps Core Community Team with other internal teams |
| ✓ Clear mental models:<br>a. Community Portal = manage MY community<br>b. Company Portal = analyze ALL communities | ✗ Duplicate features increases costs, storage needs, and time-to-market. |
| ✓ Eliminates feature redundancy, reduces engg. maintenance & optimizes backend storage | ✗ High Cognitive Load: Forces managing communities via an internal operations interface. |
| ✓ Speeds up launching the Community Portal first | |
| ✗ Some future internal-only workflows may still need Company Portal access, which could add complexity later | |

> Seeing the side-by-side comparison helped the engineering lead realize that by changing where the Core Community Team logged in, we could avoid duplicate auth, role management, and data access work, saving ~25–30% of engineering effort (about 3–4 weeks).

The engineering lead's only concern was that future internal tools might make Community Portal's codebase complex. But the only real difference we found was hiding the billing tab for Abulé's own community, which was easy to handle. So we moved ahead with the Community Portal.

BUILDING THE IA

## Defining the roles

With the architecture settled, I needed to define who could do what inside the Community Portal. Through the founder's sales conversations with potential community organizers, a clear picture emerged of the access levels needed. Six roles. In order from most to least access:

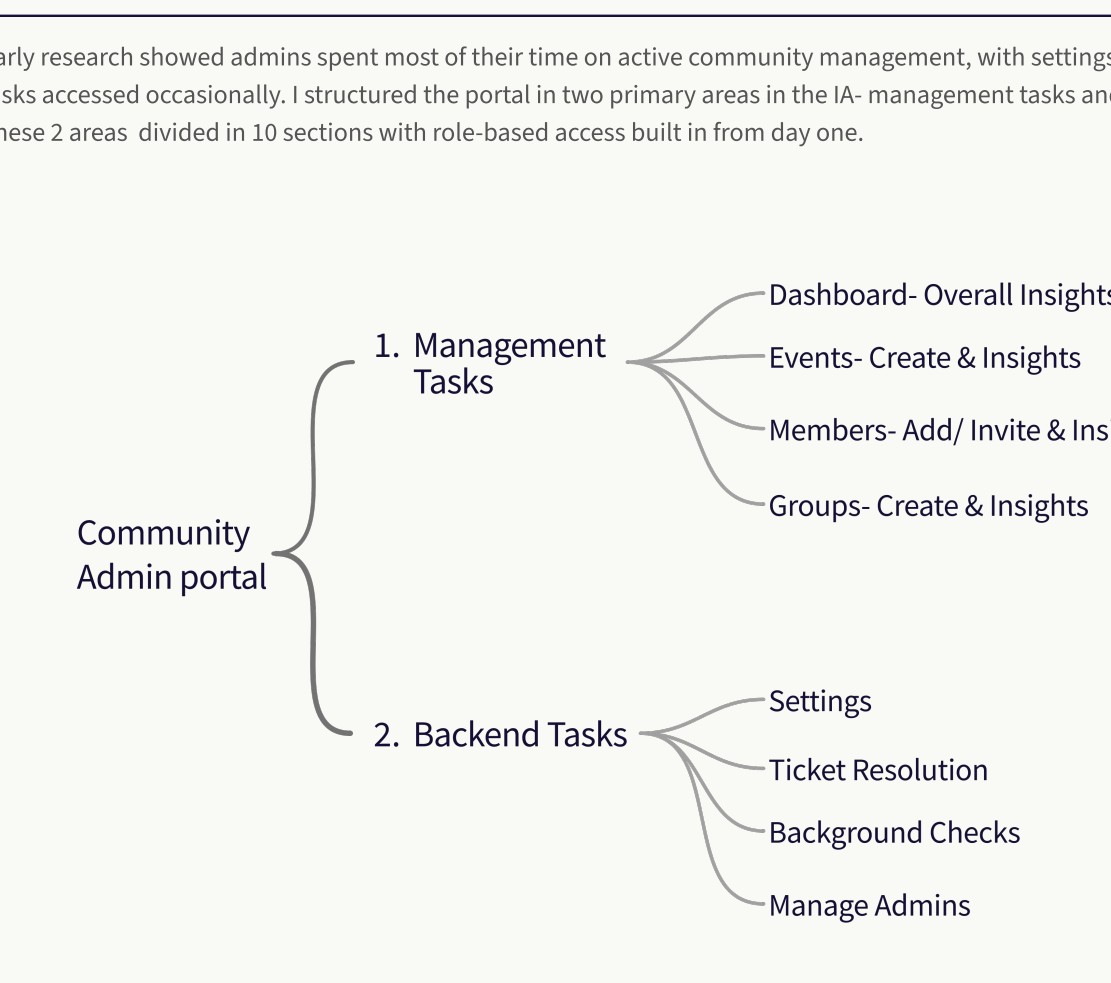
Owner → Super Admin → Admin → Community Mod. → Group Admin → Group Mod

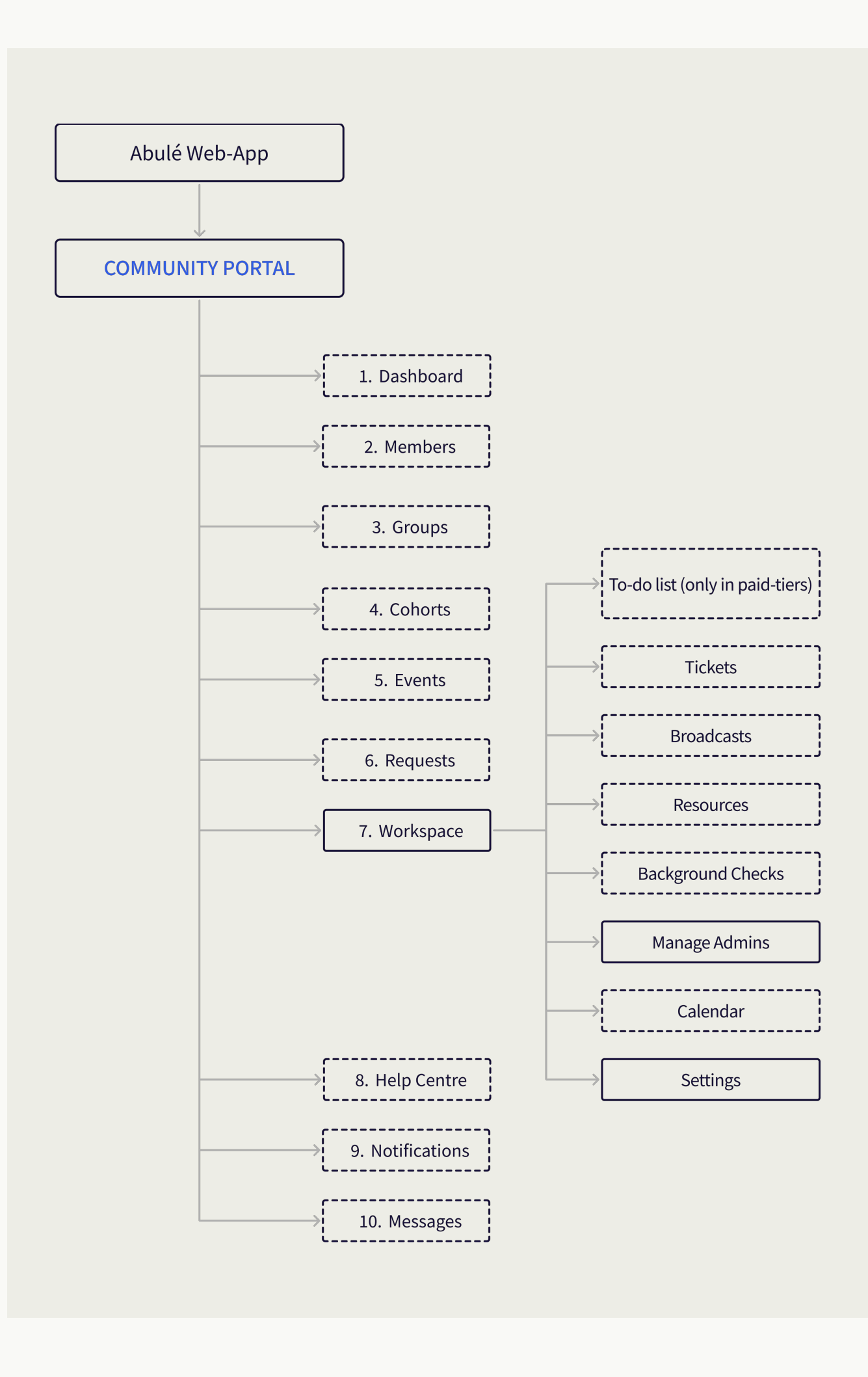
Three governance principles:

**Inheritance:** higher roles automatically include all lower role permissions
**Scope-based access:** community-wide vs. group-specific permissions are distinct
**At or below level:** admins can only manage roles beneath their own

## Structuring the IA

Early research showed admins spent most of their time on active community management, with settings and backend tasks accessed occasionally. I structured the portal in two primary areas in the IA- management tasks and backend tasks. These 2 areas divided in 10 sections with role-based access built in from day one.


Community Admin portal
1. Management Tasks
Dashboard- Overall Insights
Events- Create & Insights
Members- Add/ Invite & Insights
Groups- Create & Insights
2. Backend Tasks
Settings
Ticket Resolution
Background Checks
Manage Admins

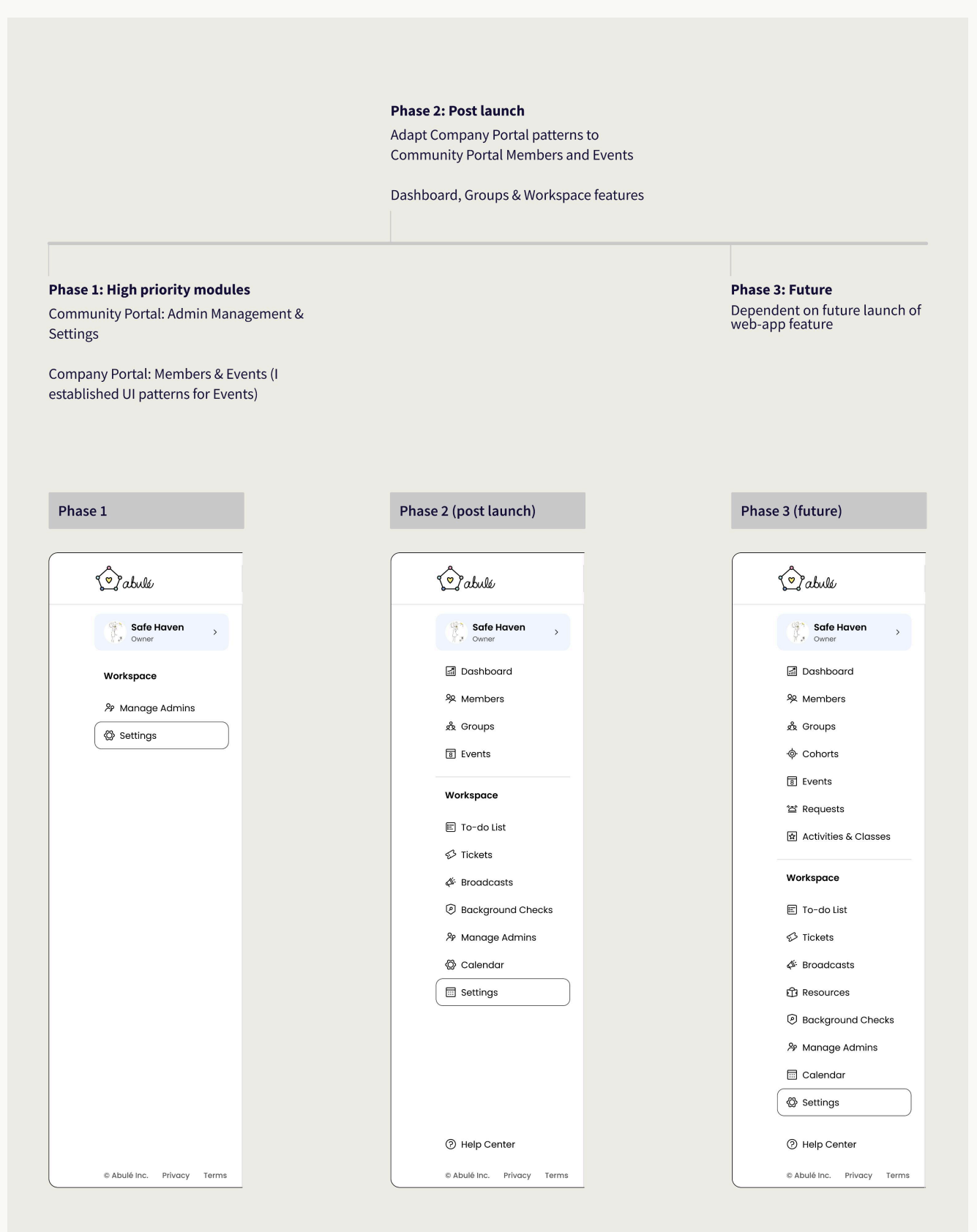

Abulé Web-App
COMMUNITY PORTAL
1. Dashboard
2. Members
3. Groups
4. Cohorts
5. Events
6. Requests
7. Workspace
8. Help Centre
9. Notifications
10. Messages
To-do list (only in paid-tiers)
Tickets
Broadcasts
Resources
Background Checks
Manage Admins
Calendar
Settings

## Phased Roll-Out strategy

I aligned the founder and the engineering lead on this structure. Their reaction was immediate:

"We can't build all of this in <2 months!"

> Designing all 10 sections with 6 role levels was the ideal solution but building it end-to-end would have taken far longer than our timeline allowed. So, we agreed to design the full structure upfront but ship it in phases.


**Phase 1: High priority modules**
Community Portal: Admin Management & Settings

Company Portal: Members & Events (I established UI patterns for Events)

**Phase 2: Post launch**
Adapt Company Portal patterns to Community Portal Members and Events

Dashboard, Groups & Workspace features

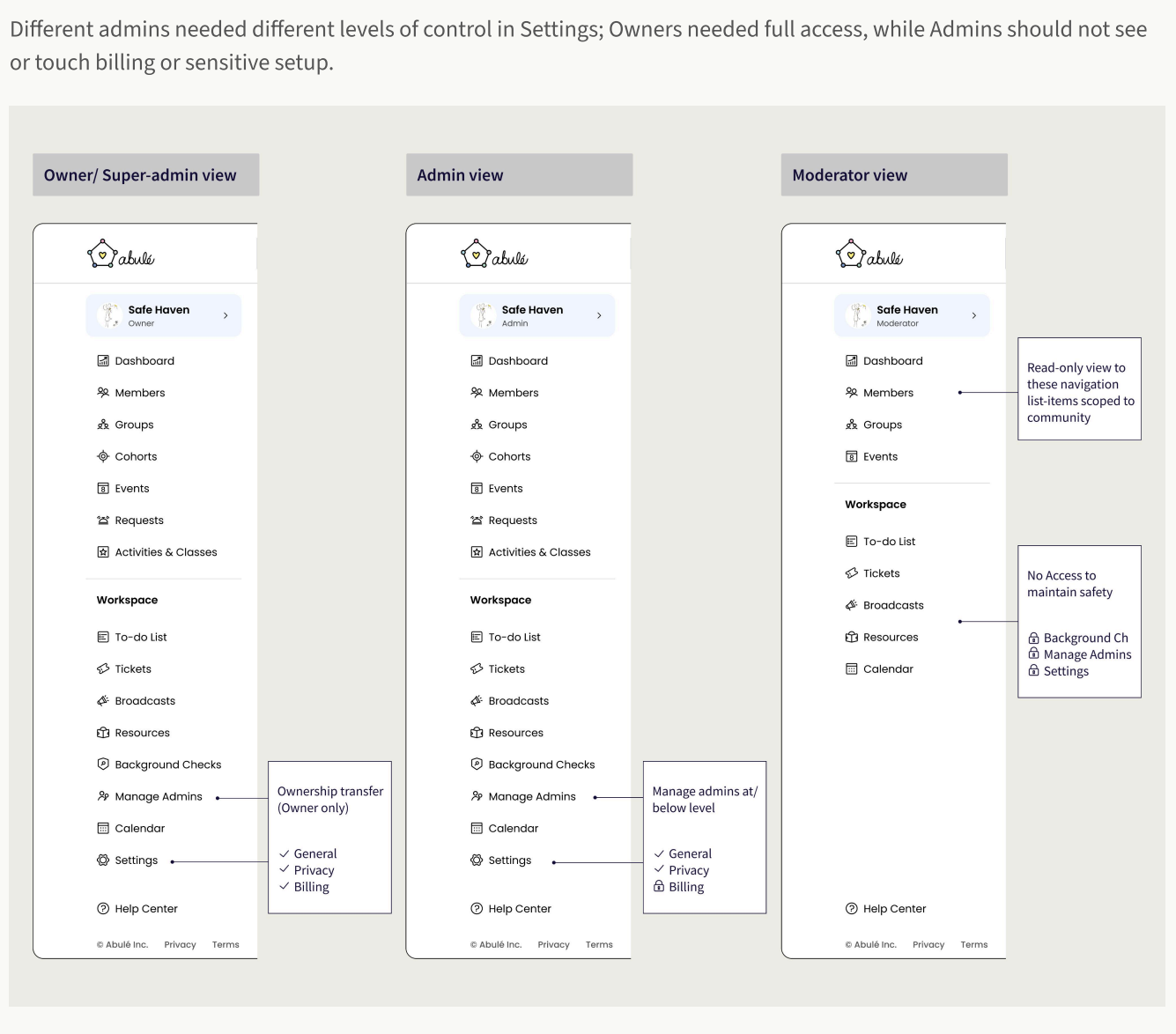
**Phase 3: Future**
Dependent on future launch of web app feature

Phase 1
Phase 2 (post launch)
Phase 3 (future)

WHAT I DESIGNED

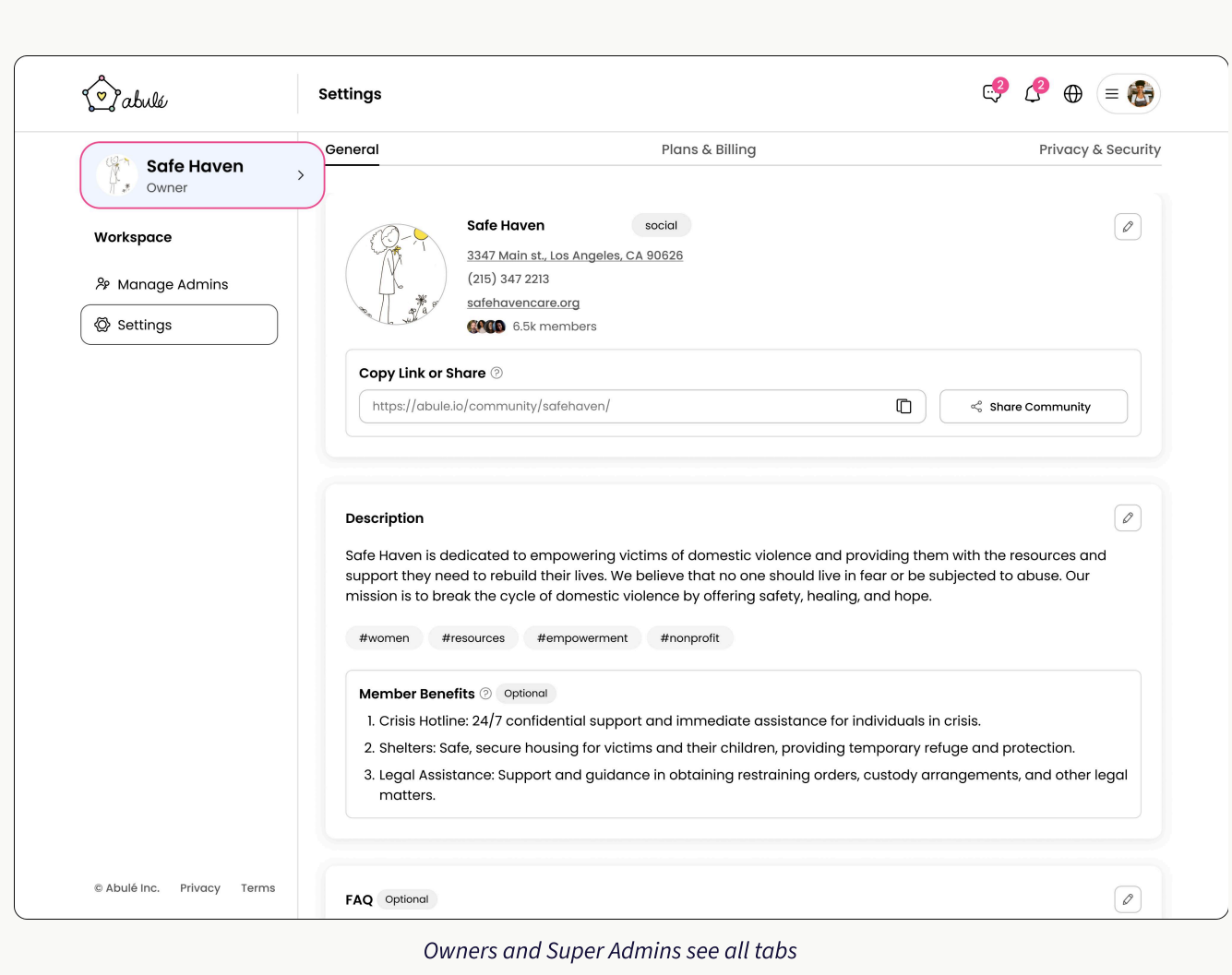
## Roll-Based Access

Different admins needed different levels of control in Settings; Owners needed full access, while Admins should not see or touch billing or sensitive setup.


Owner/ Super-admin view
Admin view
Moderator view
Ownership transfer (Owner only)
General
Privacy
Billing
Manage admins at below level
Read only view to those navigation list items scoped to community
No Access to maintain safety
Background Checks
Manage Admins
Settings

## Community Portal: Settings

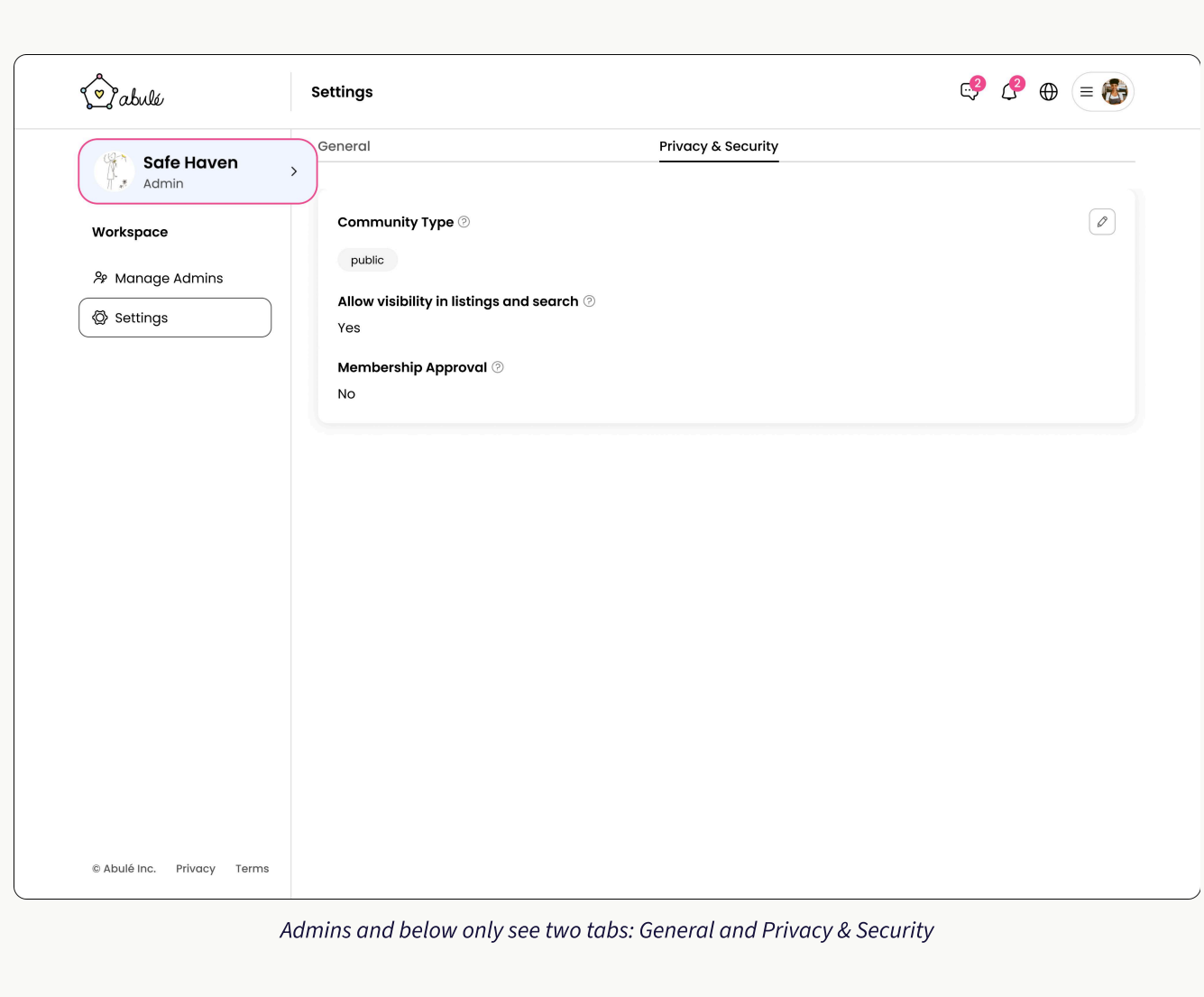
I reused the web app design system to speed up navigation design. It reduced effort and made the transition to the portal feel familiar. To support scalability and role-based access from day one, I structured Settings into three tabs:


Settings
Safe Haven
Owner
Workspace
Manage Admins
Settings
General
Plans & Billing
Privacy & Security
Copy Link or Share
Share Community
Description
Member Benefits
FAQ

Safe Haven is dedicated to empowering victims of domestic violence and providing them with the resources and support they need to rebuild their lives. We believe that no one should live in fear or be subjected to abuse. Our mission is to break the cycle of domestic violence by offering safety, healing, and hope.

1. Crisis Hotline: 24/7 confidential support and immediate assistance for individuals in crisis.
2. Shelters: Safe, secure housing for victims and their children, providing temporary refuge and protection.
3. Legal Assistance: Support and guidance in obtaining restraining orders, custody arrangements, and other legal matters.

*Owners and Super Admins see all tabs*


Settings
Safe Haven
Admin
Workspace
Manage Admins
Settings
General
Privacy & Security

Community Type: public
Allow visibility in listings and search: Yes
Membership Approval: No

*Admins and below only see two tabs: General and Privacy & Security*

*settings-prototype.mov*

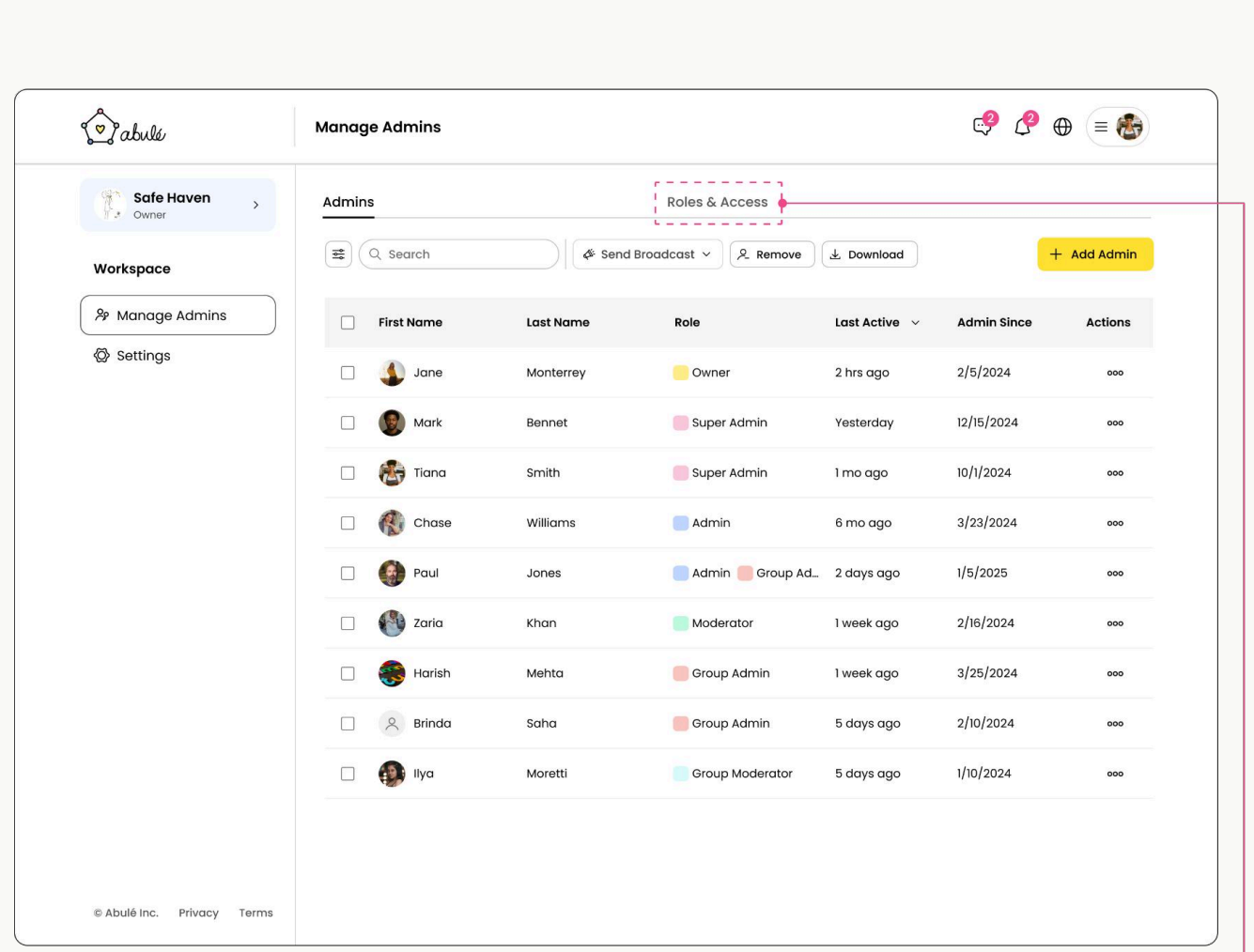
## Community Portal: Admin Management

The highest-risk section. A misclick could accidentally change someone's permissions or remove someone's access entirely. I split it into two tabs to reduce that risk:

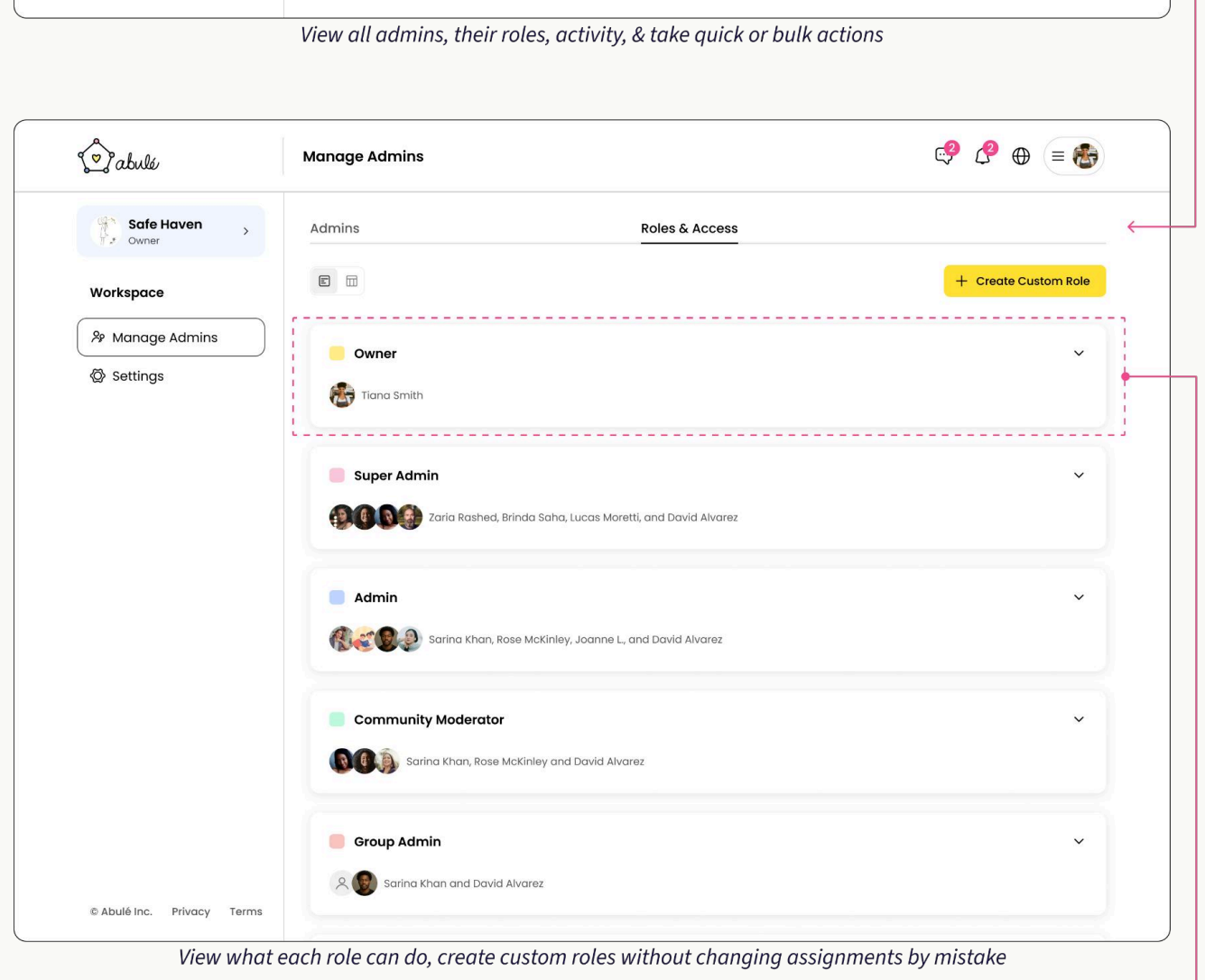
1. **Admins tab:** view all admins, their roles and activity, take quick or bulk actions, add new admins
2. **Roles and Access tab:** understand what each role can do, create custom roles in paid tiers

Three guardrails built into every interaction:

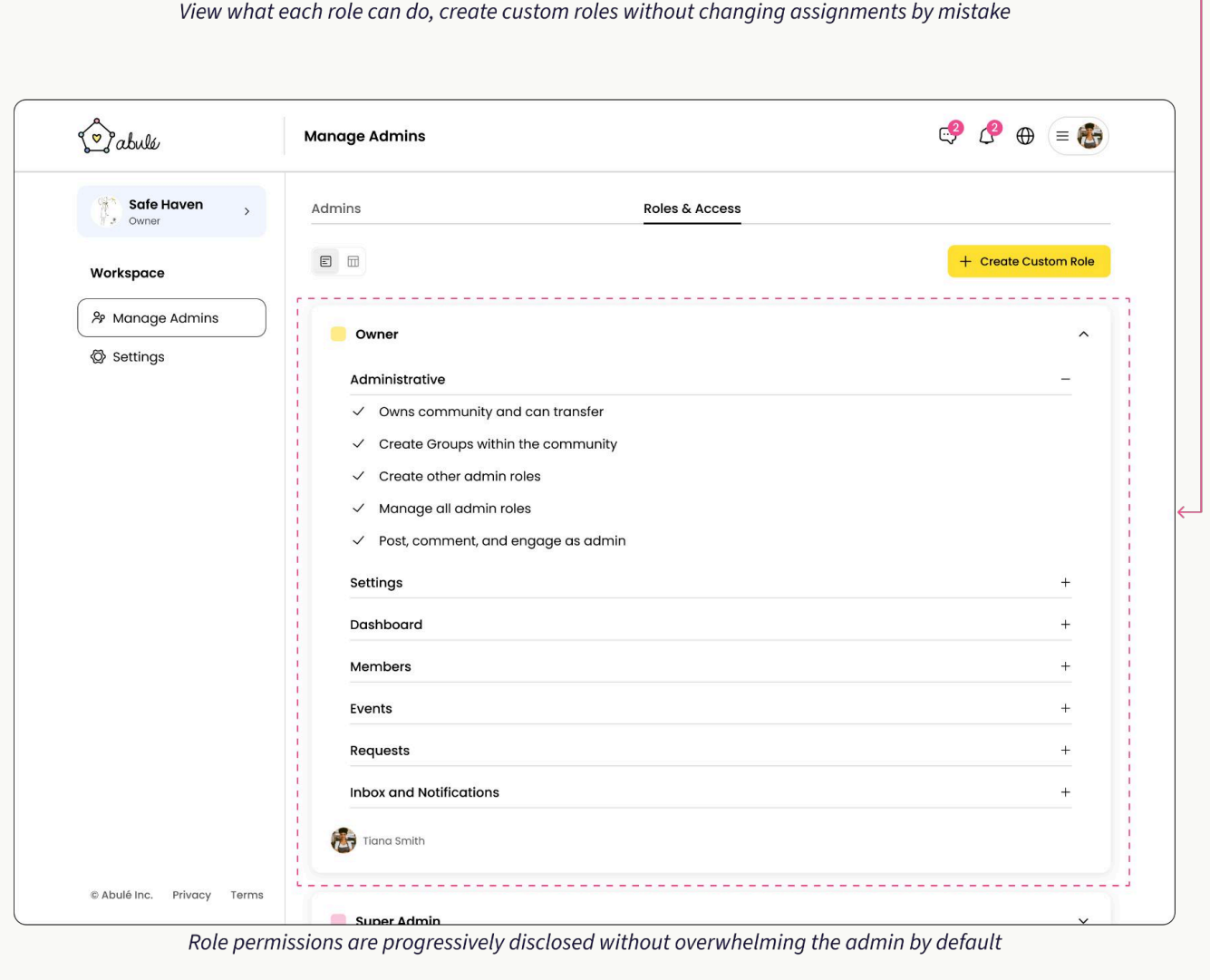
Action menus adapt by hierarchy — you physically cannot edit or remove someone above your level
Role changes trigger a step-by-step confirmation showing exactly what permissions are being added or removed
Roles are colour-coded for quick recognition so admins can scan without reading every label


Manage Admins
Admins
Roles & Access
+ Add Admin

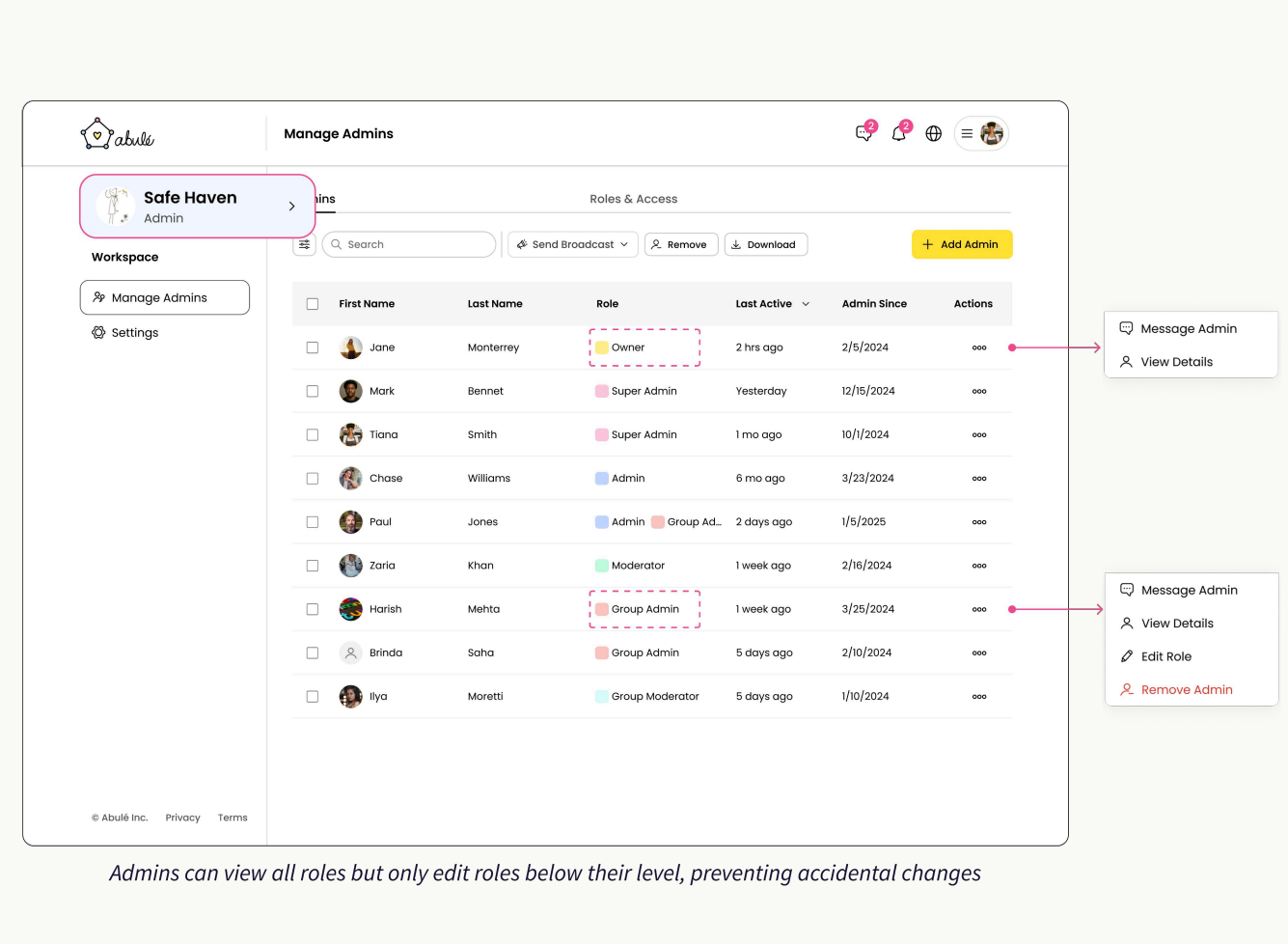
*View all admins, their roles, activity, & take quick or bulk actions*


Manage Admins
Roles & Access
+ Create Custom Role
Owner
Super Admin
Admin
Community Moderator
Group Admin

*View what each role can do, create custom roles without changing assignments by mistake*


Manage Admins
Roles & Access
+ Create Custom Role
Owner
Administrative
Owns community and can transfer
Create Groups within the community
Create other admin roles
Manage all admin roles
Post, comment, and engage as admin
Settings
Dashboard
Members
Events
Requests
Inbox and Notifications

*Role permissions are progressively disclosed without overwhelming the admin by default*


Manage Admins
Safe Haven
Admin
Message Admin
View Details
Edit Role
Remove Admin

*Admins can view all roles but only edit roles below their level, preventing accidental changes*

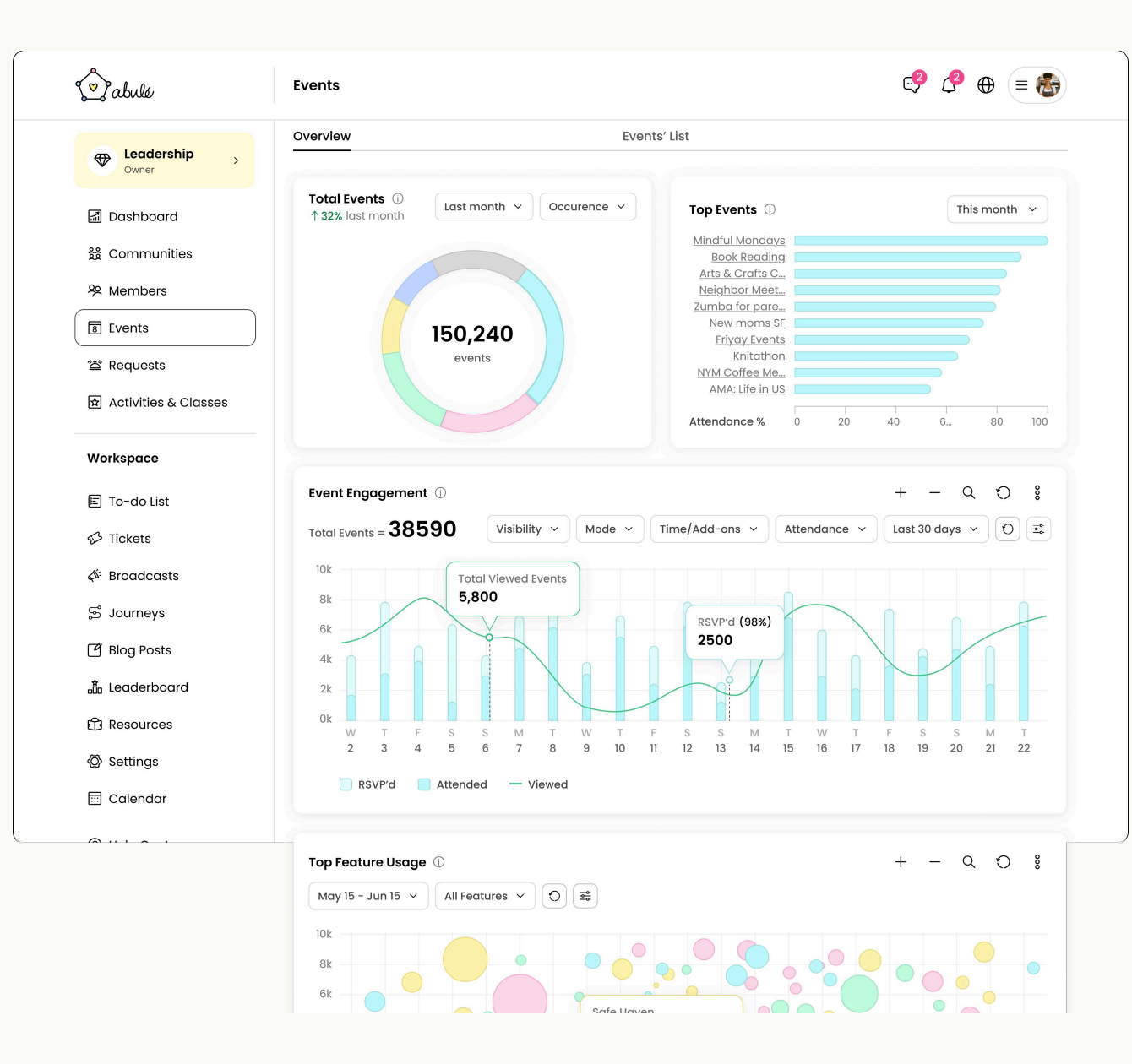
*Edit-Add-Admin-prototype.mov*

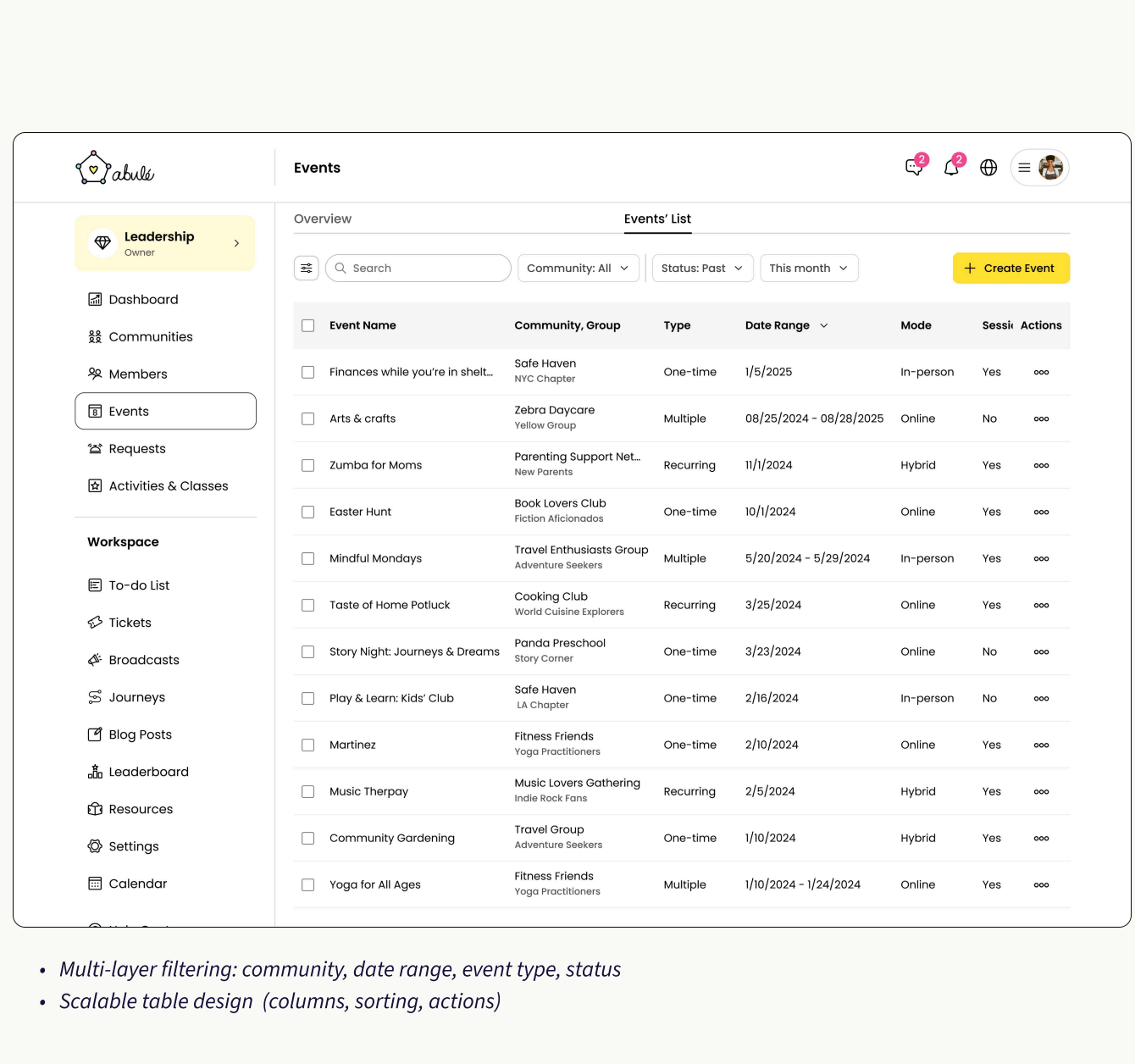
## Company Portal: Events Dashboard

As part of the platform architecture strategy, I designed the Company Portal's Events dashboard to establish UI patterns for the most data-intensive shared area. I designed these components to work for both aggregated (Company) and localized (Community) data, ensuring consistency while reducing future design and engineering effort.

Key patterns established here:

- Overview tab for analytics, Events List tab for management
- Click charts to drill into event details


Events
Leadership
Owner
Overview
Events' List
Total Events
150,240
events
Top Events
Event Engagement
Total Events • 38590
Total Viewed Events 5,800
RSVP'd
Attended
Viewed
Top Feature Usage


Events
Overview
Events' List
Search
Community: All
Status: Past
This month
+ Create Event
Event Name
Community, Group
Type
Date Range
Mode
Actions

- *Multi-layer filtering: community, date range, event type, status*
- *Scalable table design (columns, sorting, actions)*

OUTCOME

Delivered in 2 months: 65+ screens across both portals, and full platform architecture.
For the Community Portal, I delivered the IA across 6 hierarchical roles with a phased rollout plan, extended the design system and engineering handoff documentation.

## What this unlocked:

### Self-service setup

Community creators can handle billing and manage admins independently, eliminating support bottlenecks as the platform scales

### 25-30% dev time saved

Placing Core Team in Community Portal & reusing web-app patterns gave engineering ~3-4 weeks to focus on member-facing features.

### Scalable design

A six-level role system supports everything from solo owners to large admin teams without complex structural changes.

## Learning and Reflections:

**In 0→1 work, getting stakeholder alignment on the problem is harder than designing the solution.** I spent the first weeks mapping user groups and architecture before touching UI, which did create urgency but helped me turn ambiguity into clarity.

**Designing for launch while keeping things scalable was the hardest part.** I built only what Phase 1 needed, but defined the full structure upfront so future features could be added without reworking the whole system.

**Bringing engineering into conversations early saved time.** Asking questions about feasibility upfront helped shape better, more realistic solutions, faster.